DOCUMENT 3 OF 3 · LOW LEVEL DESIGN

Component and Data Design

Low Level Design. C4 level 3 components, database schema, API contracts, and the detailed design of the rules engine.

Version 0.1 · Draft for engineering review

Implements the High Level Design, version 0.1

14 September 2026

SECTION 1

C3 — Backend components

Only one container warrants decomposition for an MVP. The mobile app and dashboard are thin at this stage, so drawing their internals would add nothing.

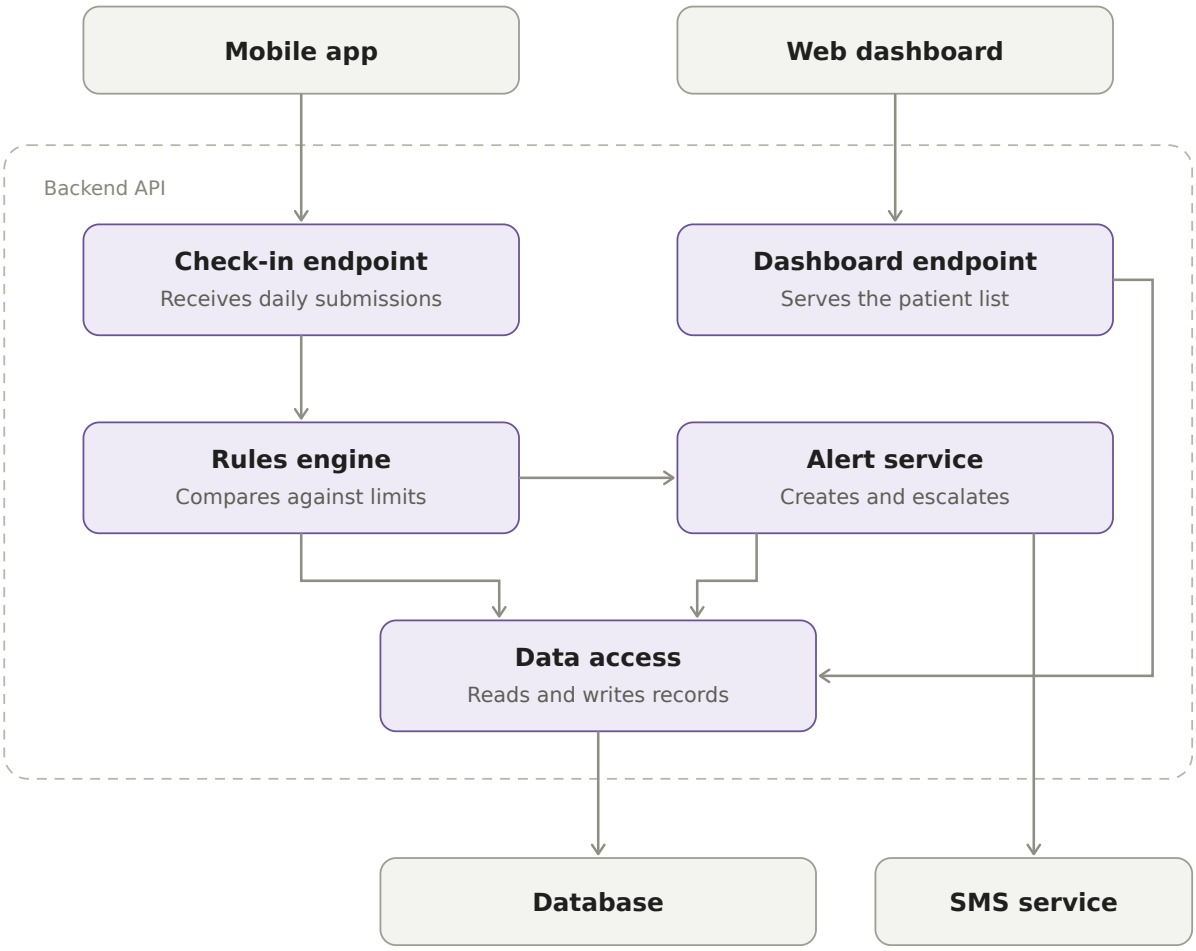


Figure 1. C4 level 3 — components inside the backend API.

Component	Responsibility	Design note
Check-in endpoint	Authenticates the patient, validates the payload, persists the reading, invokes the rules engine synchronously, returns tier-driven client instructions.	Synchronous evaluation is deliberate. The device must know immediately whether to show emergency guidance.
Dashboard endpoint	Returns the patient list ordered by tier then alert age; serves per-patient trend detail and signed photograph URLs.	Sorting lives here, not in the frontend, so ordering cannot drift between views.
Rules engine	Takes a reading, the patient's window and a ruleset; returns a tier and ranked reasons.	A pure function with no input or output. The one component a clinician may audit and the one requiring exhaustive tests.
Alert service	Creates alert records, suppresses duplicates, dispatches notifications, tracks acknowledgement state, performs escalation.	Deduplication matters more than it appears. Without it a persistent fever re-alerts daily and the queue becomes noise.
Data access	Repository layer over PostgreSQL for all reads and writes, including window construction.	Prevents the engine and the dashboard queries from each inventing their own SQL, which is what bites when a second procedure arrives.

SECTION 2

Database schema

```
create table clinics (
  id            uuid primary key,
  name          text not null,
  timezone      text not null default 'Asia/Kolkata',
  ack_window_s  int  not null default 900       -- escalation timer
);

create table clinicians (
  id            uuid primary key,
  clinic_id     uuid not null references clinics(id),
  full_name     text not null,
  email         text not null unique,
  phone_e164    text not null,
  role          text not null,                  -- nurse | surgeon | admin
  on_call       boolean not null default false,
  backup_for    uuid references clinicians(id),
  active        boolean not null default true
);

create table rulesets (
  id             uuid primary key,
  procedure_code text not null,                 -- 'TKR'
  version        text not null,                 -- semantic, immutable once approved
  config         jsonb not null,                -- see section 4
  approved_by    uuid references clinicians(id),
  approved_at    timestamptz,
  active         boolean not null default false,
  unique (procedure_code, version)
);

create table patients (
  id             uuid primary key,
  clinic_id      uuid not null references clinics(id),
  external_ref   text,                          -- hospital MRN, typed in manually
  full_name      text not null,
  phone_e164     text not null,
  language       text not null default 'en',    -- en | ta | hi
  procedure_code text not null default 'TKR',
  surgery_date   date not null,
  discharge_date date not null,
  ruleset_id     uuid not null references rulesets(id),
  threshold_overrides jsonb,                    -- per-patient surgeon adjustments
  consent_version text not null,
  consent_at     timestamptz not null,
  enrolled_by    uuid references clinicians(id),
  status         text not null default 'active',-- active | completed | withdrawn
  created_at     timestamptz not null default now()
);
create index on patients (clinic_id, status);

create table baselines (
  patient_id       uuid primary key references patients(id),
  temp_c           numeric(3,1),
  hr               int,
  spo2             int,
  pain             int,
  knee_flexion_deg int,
  walk_metres      int,
```

```
  walking_aid      text,
  recorded_at      timestamptz not null
);

create table readings (
  id                   uuid primary key,
  patient_id           uuid not null references patients(id),
  post_op_day          int  not null,
  submitted_at         timestamptz not null default now(),
  client_captured_at   timestamptz,            -- preserved across offline queue
  source               text not null default 'patient',  -- patient | proxy | nurse | synthetic
  temp_c               numeric(3,1),
  hr                   int,
  spo2                 int,
  pain                 int,
  steps                int,
  calf_pain            boolean,
  breathless           boolean,
  chest_pain           boolean,
  drainage             boolean,
  redness_spreading    boolean,
  chills               boolean,
  weight_bearing_worse boolean,
  knee_bend_proxy      text,                   -- chair | car | bike
  can_lay_flat         boolean,
  walking_aid          text,
  anticoagulant_taken  boolean,
  photo_key            text                    -- object storage key, not a URL
);
create index on readings (patient_id, post_op_day, submitted_at desc);

create table evaluations (
  id              uuid primary key,
  reading_id      uuid references readings(id),
  patient_id      uuid not null references patients(id),
  post_op_day     int  not null,
  tier            text not null,               -- normal | review | critical | emergency
  score           int,
  reasons         jsonb not null,              -- ranked [{name, score, weight, detail}]
  reason_key      text not null,               -- hash of sorted reason names, for dedup
  coverage        numeric(3,2),                -- fraction of expected fields present
  is_override     boolean not null default false,
  ruleset_version text not null,               -- makes the decision reconstructable
  evaluated_at    timestamptz not null default now()
);
create index on evaluations (patient_id, evaluated_at desc);

create table alerts (
  id              uuid primary key,
  patient_id      uuid not null references patients(id),
  evaluation_id   uuid not null references evaluations(id),
  tier            text not null,
  reason_key      text not null,
  status          text not null default 'open', -- open | acknowledged | resolved | expired
  created_at      timestamptz not null default now(),
  notified_at     timestamptz,
  acknowledged_at timestamptz,
  acknowledged_by uuid references clinicians(id),
  escalated_at    timestamptz,
  escalated_to    uuid references clinicians(id),
  resolved_at     timestamptz,
  resolved_by     uuid references clinicians(id),
  outcome         text,                        -- reassured | plan_adjusted | same_day_review |
emergency
```

```
  outcome_note    text
);
create index on alerts (patient_id, status);
create index on alerts (status, tier, notified_at);   -- drives the escalation sweep

create table notifications (
  id           uuid primary key,
  alert_id     uuid not null references alerts(id),
  clinician_id uuid not null references clinicians(id),
  channel      text not null,                   -- push | sms
  provider_ref text,
  status       text not null,                   -- queued | sent | failed
  attempts     int not null default 0,
  sent_at      timestamptz
);

create table audit_log (
  id          bigserial primary key,
  actor_type  text not null,                    -- patient | clinician | system
  actor_id    uuid,
  action      text not null,
  entity      text not null,
  entity_id   uuid,
  detail      jsonb,
  occurred_at timestamptz not null default now()
);
```

TWO SCHEMA DECISIONS WORTH DEFENDING

Readings are append-only with no uniqueness on post-operative day. The evaluator uses the latest submission for a day, but earlier ones are retained. A patient correcting a mistyped temperature must not erase the audit trail.

Every evaluation stores `ruleset_version`. This is what allows any past decision to be reconstructed after thresholds change, which will be asked for the first time a clinician disputes an alert.

SECTION 3

API contracts

Patient-facing

Endpoint	Purpose
`POST /v1/auth/otp/request`	Send a one-time passcode to the enrolled number.
`POST /v1/auth/otp/verify`	Exchange the passcode for a scoped token.
`GET /v1/me/checkin/schema`	Returns the field set active for today's post-operative day.
`POST /v1/photos/upload-url`	Returns a short-lived signed upload URL and object key.
`POST /v1/checkins`	Submit a check-in. Evaluated synchronously.

POST /v1/checkins

```
// request
{
  "post_op_day": 9,
  "client_captured_at": "2026-09-14T08:12:00+05:30",
  "idempotency_key": "b1f3...",         // survives offline replay
  "values": {
    "temp_c": 37.8, "hr": 96, "pain": 7, "steps": 150,
    "calf_pain": false, "breathless": false, "chest_pain": false,
    "drainage": true, "redness_spreading": true,
    "chills": false, "weight_bearing_worse": false,
    "anticoagulant_taken": true
  },
  "photo_key": "wounds/9f2a/day09.jpg"   // optional
}

// response 201 — note: no score, no tier name
{
  "accepted": true,
  "next_checkin": "2026-09-15",
  "client_action": "none",               // none | show_emergency_guidance
  "message": "Thanks. Your team has your readings."
}
```

The response deliberately carries no tier, score or alert status. `client_action` is the only risk-bearing field, and it takes the emergency value only at emergency tier. Omitted fields in `values` are recorded as unknown; an empty object is a valid submission and produces a low-coverage evaluation.

Clinician-facing

Endpoint	Purpose
`POST /v1/clinic/patients`	Enrol a patient with baseline, consent record and ruleset assignment.
`GET /v1/clinic/patients`	Risk-sorted list. Server-determined ordering.
`GET /v1/clinic/patients/{id}`	Detail: trends against baseline, photograph series, alert history.
`POST /v1/clinic/alerts/{id}/acknowledge`	Stops the escalation timer. Records actor and time.
`POST /v1/clinic/alerts/{id}/resolve`	Requires one of four outcomes. Closes the alert.

GET /v1/clinic/patients

```
{
  "patients": [
    {
      "id": "9f2a...", "name": "R. Nair", "post_op_day": 3,
      "tier": "critical",
      "reasons": ["Fever returned after settling", "Redness spreading"],
      "alert_id": "44c1...", "alert_age_s": 420,
      "acknowledged": false
    },
    {
      "id": "7b10...", "name": "M. Thomas", "post_op_day": 9,
      "tier": "missed",
      "reasons": ["No check-in for 2 expected cycles"],
      "alert_id": null, "acknowledged": false
    }
  ]
}
```

Ordering is `tier desc, alert_age desc`, applied server-side. Reasons are rendered clinical strings resolved from the ruleset, not internal signal names, and the numeric score is never returned to any client.

POST /v1/clinic/alerts/{id}/resolve

```
{
  "outcome": "same_day_review",   // reassured | plan_adjusted | same_day_review | emergency
  "note": "Reviewed by phone, wound photographed again, booked 4pm slot."
}
```

Returns `409` if the alert is already resolved, and `422` if the outcome is absent. There is no dismiss operation: every alert reaches an outcome.

SECTION 4

Rules engine

4.1 Data structures

```
@dataclass(frozen=True)
class Signal:
    name: str            # 'temperature', 'fever_returned'
    score: int | None    # 0-3, or None when not assessable
    weight: int
    detail: str          # rendered clinical phrase for the nurse

@dataclass(frozen=True)
class Window:
    day: int                      # post-operative day
    baseline: dict
    series: dict[str, list]       # indexed BY POST-OP DAY, None for gaps

    def rolling_mean(self, field, days, min_obs=2) -> float | None: ...
    def days_normal_before_now(self, field, rule) -> int: ...
    def abnormal_streak(self, field, rule) -> int: ...
    def personal_best(self, field) -> float | None: ...
    def monotonic_rise(self, field, days) -> float | None: ...
    def coverage(self, days) -> float: ...

@dataclass(frozen=True)
class Result:
    tier: str
    reasons: list[Signal]
    score: int | None
    is_override: bool
    ruleset_version: str
```

THE SINGLE MOST IMPORTANT IMPLEMENTATION DETAIL

`Window.series` is indexed by post-operative day with an explicit `None` for gaps — never by "the last three submissions". Otherwise a patient who skips a week has day 9 silently compared against day 2, and every trend detector quietly lies.

4.2 Configuration schema

All clinical behaviour is expressed here. Every input, binary or continuous, normalises to an integer 0 to 3 before anything downstream touches it, which is what makes them comparable.

```
version: "1.3.0"
procedure_code: TKR
tiers: { review_at: 5, critical_at: 12 }
max_points_per_parameter: 6

rules:
  - name: pain
    parameter: pain
    weight: 2
    bands_by_day:                        # the expected recovery corridor
      "0-3":   [[0,6,0], [6,8,1], [8,9,2],  [9,10,3]]
      "4-10":  [[0,4,0], [4,6,1], [6,8,2],  [8,10,3]]
      "11-42": [[0,3,0], [3,5,1], [5,7,2],  [7,10,3]]

  - name: temperature
```

```
    parameter: temp_c
    weight: 3
    bands: [[0,37.5,0], [37.5,38.0,1], [38.0,38.5,2], [38.5,99,3]]

  - name: heart_rate
    parameter: hr
    weight: 2
    relative_to_baseline: true          # banded on delta, not absolute
    bands: [[0,10,0], [10,20,1], [20,30,2], [30,99,3]]

  - name: calf_pain
    parameter: calf_pain
    weight: 3
    binary_score: 3                     # reaches band 3, so it overrides

  - name: wound_drainage
    parameter: drainage
    weight: 2
    active_from_day: 5                  # silent before this
    binary_score: 2

trends:
  - name: pain_rising
    parameter: pain
    detector: delta_vs_rolling_mean
    window_days: 3
    min_observations: 2
    weight: 2
    bands: [[0,2,0], [2,3,1], [3,4,2], [4,10,3]]

  - name: fever_returned                # highest-yield detector in the set
    parameter: temp_c
    detector: reemergence
    normal_days_required: 2
    weight: 3
    score: 3

  - name: slow_burn
    parameter: temp_c
    detector: monotonic_rise
    window_days: 3
    min_total_rise: 0.6
    weight: 2
    score: 2

  - name: mobility_regression
    parameter: steps
    detector: drop_from_personal_best
    drop_fraction: 0.5
    weight: 1
    score: 2

  - name: persistent_abnormal
    parameter: any
    detector: streak
    min_days: 3
    weight: 1
    score: 1

overrides:
  emergency: [breathless, chest_pain]    # bypass scoring entirely
```

4.3 Evaluation pipeline

```
def evaluate(reading, window, config) -> Result:
    threshold_signals = score_thresholds(reading, window, config)
    trend_signals     = score_trends(reading, window, config, threshold_signals)

    signals = cap_per_parameter(
        threshold_signals + trend_signals,
        config.max_points_per_parameter,
    )

    # Stage 1 — overrides bypass scoring
    if reading.breathless or reading.chest_pain:
        return Result(EMERGENCY, [Signal("pe_risk", 3, 0,
               "Possible clot on the lung")], None, True, config.version)

    if top := [s for s in signals if s.score == 3]:
        return Result(CRITICAL, top, None, True, config.version)

    # Stage 2 — weighted sum of everything remaining
    total = sum(s.score * s.weight for s in signals if s.score)
    tier  = (CRITICAL if total >= config.tiers.critical_at
             else REVIEW if total >= config.tiers.review_at
             else NORMAL)

    return Result(tier, rank_by_contribution(signals), total, False, config.version)
```

Two things to note. `score_trends` receives the threshold signals, because detectors such as `persistent_abnormal` need to know what is currently abnormal. And trend signals reaching band 3 escalate through the same override loop as threshold signals, with no special casing anywhere.

4.4 Worked example

The same patient profile on two different days, with `review_at: 5` and `critical_at: 12`.

Day 9 — pain 7 against a three-day mean of 4, temperature 37.8, heart rate 96 on a baseline of 70, redness spreading, steps down from 500 to 150

Signal	Band	Weight	Points
Pain	2	2	4
Temperature	1	3	3
Heart rate (delta 26)	1	2	2
Redness spreading	2	3	6
Mobility regression	1	1	1
Pain rising vs trend	1	2	2
		Total	**18 → critical**

No single signal reached band 3. This is the combination case escalating correctly, and it is why a pure override model is insufficient.

Day 2 — identical pain of 7 and temperature of 37.8, nothing else abnormal

Pain bands to 0 on the day 0 to 3 corridor. Temperature scores 1, weighted to 3. No trend bonus, because there is no improving trajectory yet to break. **Total 3 → normal.** Correct: that is an unremarkable day-two picture and should wake nobody.

4.5 Rules that prevent false confidence

- **A detector below its `min_observations` emits nothing at all**, never a zero. Zero means assessed and normal, which is a lie during warm-up and would mask real problems in the first days.
- **Missing data is never scored.** A skipped oxygen reading is unknown, not reassuring. Coverage is tracked separately and a low-coverage evaluation is marked low-confidence.
- **Contribution is capped per parameter.** A high-and-rising pain reading scores twice, on its band and on its trend. That is usually correct but compounds quickly; the cap keeps the highest-scoring signals and prevents one vital from dominating the total.
- **Hysteresis on trends.** A non-override trend signal must hold across two consecutive evaluations before it can raise the tier. Trend states persist for days and would otherwise flap across band boundaries.

SECTION 5

Alert service and worker

5.1 Alert lifecycle

State	Entered when	Exits to
`open`	Evaluation returns critical or emergency and no live duplicate exists.	`acknowledged`, or escalation (remains open).
`acknowledged`	A clinician acknowledges. Escalation timer stops.	`resolved`.
`resolved`	An outcome is recorded. Terminal.	—
`expired`	A review-tier item ages past its window without a higher tier arising.	Terminal. Recorded, not silently dropped.

5.2 Deduplication

Deduplication keys on the **reason set**, not the alert. `reason_key` is a stable hash of the sorted contributing signal names. A new evaluation producing an identical `reason_key` within twenty-four hours does not create an alert unless its tier is higher than the live one. This is what stops a persistent fever re-alerting every morning until the queue becomes wallpaper.

```
def maybe_create_alert(evaluation, patient, now):
    if evaluation.tier not in (CRITICAL, EMERGENCY):
        return None

    live = alerts.find_live(patient.id, evaluation.reason_key, within_hours=24)
    if live and TIER_RANK[evaluation.tier] <= TIER_RANK[live.tier]:
        alerts.attach_evaluation(live.id, evaluation.id)   # trail, no new alert
        return None

    alert = alerts.create(patient.id, evaluation)
    notifier.dispatch(alert, recipients=on_call_for(patient.clinic_id))
    return alert
```

5.3 Escalation sweep

```
# runs every 60 seconds
def sweep_escalations(now):
    due = alerts.select_open_critical_past_window(now)   # index-backed
    for alert in due:
        if alert.escalated_at:
            continue                                     # idempotent
        backup = backup_for(alert.notified_clinician_id)
        alerts.mark_escalated(alert.id, backup.id, now)
        notifier.dispatch(alert, recipients=[backup], reason="escalation")
```

> **THE MOST DANGEROUS FAILURE IN THE SYSTEM**
>
> If the worker stops, escalation ceases silently and nothing in the product reports it. The worker must emit a heartbeat, and a monitor *outside* the worker must alarm on its absence. A self-monitoring escalation timer is not a safety mechanism.

5.4 Missed check-in sweep

```
# runs daily, after the check-in window closes
def sweep_missed(now):
    for patient in patients.active_with_expected_checkin(now):
        if readings.exists_for_day(patient.id, patient.post_op_day(now)):
            continue
        synthetic = Reading.empty(day=patient.post_op_day(now), source="synthetic")
        window    = data_access.window_for(patient, patient.post_op_day(now))
        result    = evaluate(synthetic, window, ruleset_for(patient))
        evaluations.store(result, patient, reading=None)
        # absence detectors fire through the normal path
```

The same `evaluate()` is invoked from both the ingest path and the worker. A parallel implementation inside the worker would be a defect, because the two would diverge on the first threshold change.

SECTION 6

Client design

6.1 Day-dependent form rendering

The client renders whatever `GET /v1/me/checkin/schema` returns and holds no clinical knowledge of its own. This keeps the field schedule in the same versioned configuration as the thresholds.

```
{
  "post_op_day": 7,
  "cadence_note": null,
  "guidance": null,
  "sections": [
    { "key": "readings", "title": "Readings", "fields": [
        {"key":"temp_c","type":"number","label":"Temperature","unit":"C"},
        {"key":"hr","type":"number","label":"Pulse"},
        {"key":"spo2","type":"number","label":"Oxygen","unit":"%"},
        {"key":"pain","type":"scale","min":0,"max":10,"label":"Pain right now"}]},
    { "key": "checks", "title": "Quick checks", "fields": [
        {"key":"breathless","type":"bool","label":"New breathlessness or chest pain?",
         "emphasis":"emergency"},
        {"key":"calf_pain","type":"bool",
         "label":"Is the calf of your operated leg newly painful, or more swollen than your other
leg?",
         "illustration":"leg_calf"},
        {"key":"drainage","type":"bool","label":"Is fluid still leaking from your wound?",
         "is_new": true}]}
  ]
}
```

Rule	Reason
Inactive fields are absent, not disabled	A visible question that cannot produce an alert teaches patients that some questions do not matter.
All booleans share one polarity	"Yes" is always the concerning answer. A single reverse-scored item causes mis-taps, and it is older patients who pay for it.
No pre-selected default	Shipping with "No" pre-selected yields a fortnight of clean data from a patient with a spreading infection.
`is_new` marks first appearance	Prompts an actual read rather than tapping through from muscle memory.
Comparison anchors sit in the label	Without "than your other leg" or "than yesterday", nearly every patient answers yes.

6.2 Offline queue

Submissions are written to IndexedDB with a client-generated `idempotency_key` and the original `client_captured_at`. A service worker retries with exponential backoff on reconnection. The server treats a repeated idempotency key as a no-op returning the original response, so a replayed queue cannot create duplicate readings or duplicate alerts. Up to seven days of submissions are retained; beyond that the oldest are surfaced to the patient rather than silently dropped.

One consequence worth stating: an offline submission that would have triggered emergency guidance cannot show it until sync. The emergency questions are therefore evaluated *client-side as well*, using a static rule shipped with the app, so that a reported chest pain shows guidance immediately regardless of connectivity.

6.3 Dashboard

Polls the list endpoint every thirty seconds. Ordering and reason text are server-supplied and never recomputed client-side. Acknowledge is optimistic with rollback on failure; resolve is not, because it is the record of a clinical decision.

SECTION 7

Testing strategy

The evaluator is the component that can cause harm, and it is fully testable without a database, a device or a patient. The test strategy is weighted accordingly.

7.1 Tooling

Need	Tool	Rationale
Harness	pytest	Parametrising over fixture files yields one named test per synthetic patient.
Fixture schema	Pydantic	Already present with FastAPI. Validates fixture YAML and gives readable errors when a clinician edits one.
Fixture files	YAML	Diffable in version control and readable by a surgeon. Trajectories do not belong in Python.
Curves and noise	numpy, seeded	Piecewise linear plus gaussian noise covers every realistic trajectory. Seed every fixture explicitly.
Invariants	Hypothesis	Property tests over random trajectories, not only hand-picked ones.
Regression	syrupy	Snapshots the full evaluation trace, so a weight change appears as a reviewable diff.
Clinical review	plotly	One chart per fixture with the escalation day marked. This is the artefact shown to the surgeon.

Off-the-shelf synthetic data tools are the wrong fit: they learn a distribution from real data and resample it, whereas the requirement is to author trajectories chosen for clinical interest. The generator is roughly 150 lines and should be written. Synthea remains useful for demographics and FHIR-shaped enrolment payloads, but it produces encounter-level records and will not yield a daily post-operative temperature curve.

7.2 Fixture patients

Fixture	Exercises
Smooth recovery	The false-positive rate. Must never escalate.
Slow-burn infection	`fever_returned` and `pain_rising`. Temperature settles two days before climbing, which is precisely what arms the detector.
Sudden clot, day four	Binary override path and emergency tier.
Erratic reporter	Gap handling. Confirms the trend layer does not panic at missing days.
Drop-out at day five	The worker path and absence detection, which never runs if only the ingest path is tested.

7.3 The assertion that matters

```
@pytest.mark.parametrize("fx", load_fixtures(), ids=lambda f: f.name)
def test_first_escalation(fx, ruleset, snapshot):
    timeline, trace, first = generate(fx), [], None

    for day in range(fx.days):
        reading = timeline[day] or empty_reading(day)
```

```
        window  = Window.from_timeline(timeline[:day + 1], fx.baseline, day)
        result  = evaluate(reading, window, ruleset)
        trace.append((day, result.tier, [r.name for r in result.reasons]))
        if first is None and result.tier in (CRITICAL, EMERGENCY):
            first = day

    lo, hi = fx.expect.escalates_between
    assert first is not None, f"{fx.name} never escalated"
    assert lo <= first <= hi, f"{fx.name} escalated day {first}, wanted {lo}-{hi}"
    assert snapshot == trace
```

The range assertion is what gives the test clinical meaning. "Catches the infection between day 8 and day 11" is a claim a surgeon can agree or disagree with; "returns tier critical" is not. Snapshotting the reasons as well as the tier matters, because an engine that escalates on the right day for the wrong reason is worse than one that misses, since it will be trusted.

7.4 Properties worth asserting

1. A strictly improving trajectory never escalates.
2. Worsening any single input never lowers the tier. This catches cap and weight defects and is the highest-value property in the set.
3. Evaluation is deterministic for identical inputs and ruleset version.
4. Removing a reading never lowers the tier below what the present data alone would produce.

7.5 Continuous integration gate

Any change to the evaluator or to an active ruleset runs the full fixture suite. A change that alters any fixture's first-escalation day fails the build and requires explicit clinical re-approval, recorded as a new ruleset version. This is the mechanism that prevents a weight adjustment from quietly breaking detection three weeks downstream.